

BRANCH: Bypassing Multi-Scanner AI Guardrails

William Hackett^{1,2} Peter Garraghan^{1,2}

¹Mindgard, ²Lancaster University

{william.hackett, peter}@mindgard.ai

Abstract

AI systems increasingly rely on Large Language Models (LLMs) as core reasoning engines, making them targets for prompt injection and jailbreaks. Guardrails monitor and validate model inputs and outputs, yet their isolated, task-focused detection leaves gaps in their classification making them susceptible to bypasses. In response, *guardrail systems* formed by multiple *scanners* have emerged that collaboratively detect different types of malicious instructions, whereby shared latent representations across classification boundaries render established bypassing techniques ineffective. We propose *BRANCH*, a bypassing methodology designed for multi-scanner guardrail systems. Our method leverages a branching tree search approach that dynamically applies adversarial perturbation against individual scanners, with subsequent perturbation optimization and technique selection based on overall improvement across all guardrail system scanners, effectively decoupling bypass evaluation from attack signal optimization. Our findings demonstrate that *BRANCH* achieves 100% attack success rate across 6 guardrail systems in 120 scenarios with 72% fewer queries and 4.5x reduced wall-clock time compared to established techniques, while preserving semantic meaning within the bypass. We also show how bypasses generated by *BRANCH* transfer to 29 unseen guardrails, including 8 commercial black-box guardrails, improving attack success in some cases up to 100% with no additional optimization.

1 Introduction

The adoption of *Generative Artificial Intelligence (GenAI)* has led to the rapid proliferation of *AI systems* that use these models to perform complex tasks (Nastoska et al., 2025; Shethiya, 2023). Subsequently, AI systems have become targets for adversarial prompting techniques such as prompt injection and jailbreaking that attackers use to exfiltrate sensitive data, manipulate model behavior, or

escape trained safety alignment (Wu et al., 2024).

To mitigate these attacks, *guardrails* were created to monitor and validate inputs and outputs to GenAI models within an AI system. Guardrails typically use a range of detection techniques such as text-classification models, regular expression, and pattern analysis designed to detect specific content categories including toxicity, prompt injection, and jailbreaks (Ayyamperumal and Ge, 2024; Dong et al., 2024). An individual single-purpose guardrail, which we refer to as a *scanner* provides narrow, single-category classification that is susceptible to the evasion techniques established against text classifiers such as character injection and word substitution (Bassani et al., 2025; Hackett et al., 2025). In response, *guardrail systems* have emerged as the more sophisticated solution, whereby an ensemble of scanners are combined to detect malicious content across categories, covering blindspots in detection capabilities. This forces the attacker to bypass every scanner within the system simultaneously in order to succeed (Protect AI, 2025; Vijil AI, 2024; Chennabasappa et al., 2025).

Although guardrail systems have become commonplace within production AI systems, no research has evaluated the effectiveness of current attacks against multiple scanners simultaneously. We hypothesize that the presence of multiple scanners creates a multi-objective optimization problem for attackers, whereby the *shared latent representations* across scanners constrain established bypassing techniques in overlapping decision boundaries (Miettinen, 1999; Demontis et al., 2019). As a result, current techniques either fail to generate a bypass that evades all guardrails at once, or succeed only at substantial computational cost.

In this paper, we propose *BRANCH*, a bypassing methodology designed for bypassing guardrail systems. *BRANCH* leverages a branching tree search that chains multiple perturbation techniques together, isolating bypassing to individual scanners,

while selecting branches which gradually bypass all scanners within a guardrail system simultaneously. Our approach is applicable to any malicious instruction while being perturbation agnostic, enabling the generation of bypasses against guardrail systems comprising of multiple scanners. Furthermore, we demonstrate that bypasses generated by BRANCH also transfer to a large set of guardrails unseen during generation. To our knowledge, our work is the first approach to reliably bypass more sophisticated guardrail systems as a whole, instead of individual guardrail scanners.

We make the following contributions:

1. We investigate the effectiveness of current single-objective evasion techniques against multi-scanner guardrail systems, demonstrating their challenges and limitations when operating within multi-objective bypass scenarios, achieving lower success at substantive computational cost (Section 3).
2. We propose a novel multi-objective guardrail bypassing methodology called *BRANCH* to address the challenges of established single-objective evasion techniques, and demonstrate its ability to achieve a 100% bypass success rate across 6 target guardrail systems using 72% fewer queries and 4.5x less wall-clock time compared to the State-of-the-Art, while preserving semantic content comparable to established techniques (Section 6.1, Section 6.2).
3. We demonstrate that bypasses generated by BRANCH successfully transfer and increased Attack Success Rate by an average of 32.9 percentage points across 29 unseen production guardrails without any adversarial perturbation, including 8 commercially offered black-box guardrail systems. (Section 6.3).

2 Background

2.1 Guardrails

Guardrails are predictive systems embedded within AI systems which monitor the inputs and outputs of GenAI models, enabling an AI system to react if malicious content is detected (Ayyamperumal and Ge, 2024). We categorize guardrails into two types: *scanners*, which are individual, single-purpose guardrails designed to detect a single category of content, and *guardrail systems*, which are

ensembles of scanners that combine their outputs to make a single classification decision.

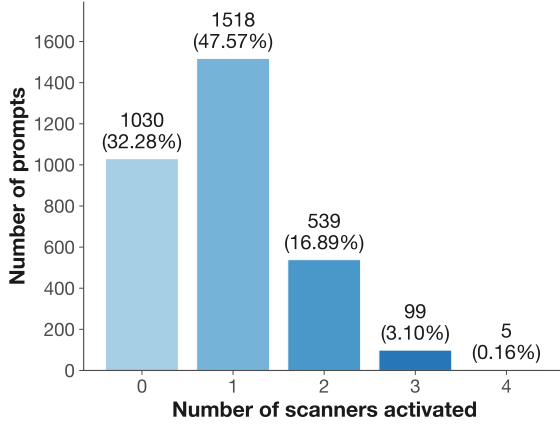
Guardrail Scanners. Designed to detect a single category of content such as toxicity, prompt injections, or jailbreaks. A scanner may be implemented through a range of detection techniques. The most common are AI-based, using text-classification models pre-trained to detect a specific category of content, while others leverage regular expression and pattern matching, or LLM-as-a-judge judge (Rebedea et al., 2023; Zheng et al., 2024). Text-classification has been widely adopted because it can be pre-trained on datasets containing examples of the content it is designed to detect, using architectures such as transformer-based models (e.g. BERT) that are fine-tuned to classify inputs as malicious or non-malicious based on the linguistic features and patterns (Li et al., 2025; Dubey and et al., 2024; Microsoft Corporation, 2024). This allows a scanner to generalize beyond its training examples to detect a wide range of unseen malicious inputs. However, because each scanner is trained to detect a single category of content, its classification is isolated and task-focused, providing no coverage of the categories it was not trained on (Hackett et al., 2025).

Guardrail Systems. These systems address scanner limitations by leveraging multiple scanners designed for different categories in tandem to create a single classification decision to provide more comprehensive coverage across threat categories (Protect AI, 2025; Vijil AI, 2024; Chennabasappa et al., 2025). This approach prevents attackers from exploiting weaknesses within the classification capabilities of a single scanner as the decision boundaries in a guardrail system are more complex. Production guardrail systems offered by major vendors such as OpenAI, Microsoft, and Google combine multiple scanners to detect malicious content across categories (Microsoft Corporation, 2024; Zeng et al., 2024; Markov et al., 2023a).

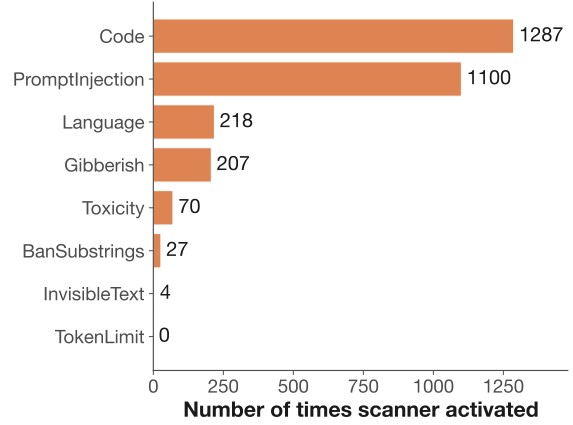
2.2 Bypassing Techniques

The goal of an attacker targeting a guardrail system is to generate a *bypass*, an input that evades the target AI system so that it is classified as non-malicious (Hackett et al., 2025). Bypasses originate from *Adversarial Machine Learning (AML) evasion*, that generates adversarial examples against image and text classifiers.

Adversarial ML Evasion. These techniques were adapted to guardrails due to their reliance



(a) Total Scanner Activations.



(b) Activations Per Scanner.

Figure 1: **Static Scanner Activations.** The number of scanners activating, and their distribution, across 8 scanners within LLM Guard during inference of 3,191 prompt injection and jailbreak samples from PIGuard (Li et al., 2025).

upon ML models (such as text classifiers) to detect malicious content. Bypassing techniques differ in how they generate perturbations. For example, genetic algorithms can be used to evolve the input (Alzantot et al., 2018), BERT-based masking to identify and replace most important tokens (Garg and Ramakrishnan, 2020; Li et al., 2020), and substituting characters with homoglyphs or intentional typos (Li et al., 2019; Jin et al., 2020). Which tokens that the technique modifies is determined by *word importance ranking*, which scores each word by the change in the guardrail’s confidence when that word is removed and prioritizes higher-impact words for modification (Jin et al., 2020; Ren et al., 2019). This enables an attacker to target features within the input which the target directly relies upon for classification, allowing these areas to be modified more effectively (Biggio et al., 2013).

Perturbation Methods. A *perturbation* is a modification applied to the input to generate a bypass (Morris et al., 2020). Perturbation methods differ in the granularity at which they are applied, which we separate into two categories (Garg and Ramakrishnan, 2020; Li et al., 2020, 2019; Gao et al., 2018). *Character injection* inserts characters into the input to evade detection, either invisible characters such as zero-width spaces, or characters that are visually similar to the originals (Gao et al., 2018). Although these perturbations are perceptible to humans, GenAI models remain capable of interpreting the underlying obfuscated or injected input. *Word substitution*, by contrast, replaces words in the input with synonyms or similar alternatives, generating substitutes through meth-

ods such as BERT-based masking (Garg and Ramakrishnan, 2020; Li et al., 2020), or word search algorithms (Bird and Loper, 2004).

3 The Challenges of Established Bypassing Techniques Against Guardrail Systems

As guardrail systems have more complex decision boundaries compared to single scanners, single-objective evasion techniques struggle when attempting to successfully bypass multiple scanners simultaneously. We attribute the challenges of these techniques to two contributing factors: *overlapping decision boundaries* between guardrails that share latent representations, and *scanner interference*, where a single-objective technique must collapse the per-scanner confidence scores into one scalar, resulting in treating all scanners as a single target, and causing guardrails which were not actively blocking to block when perturbations are applied to evade another.

3.1 Overlapping Decision Boundaries

The shared latent representations of scanners within a guardrail system can lead to overlapping decision boundaries, where the same input may be classified as malicious by multiple scanners due to similar features or patterns associated with the content each scanner is designed to detect. As scanners share these latent features, perturbations that reduce the confidence of one scanner by moving the input across a decision boundary can simultaneously increase another scanner’s confidence score, since

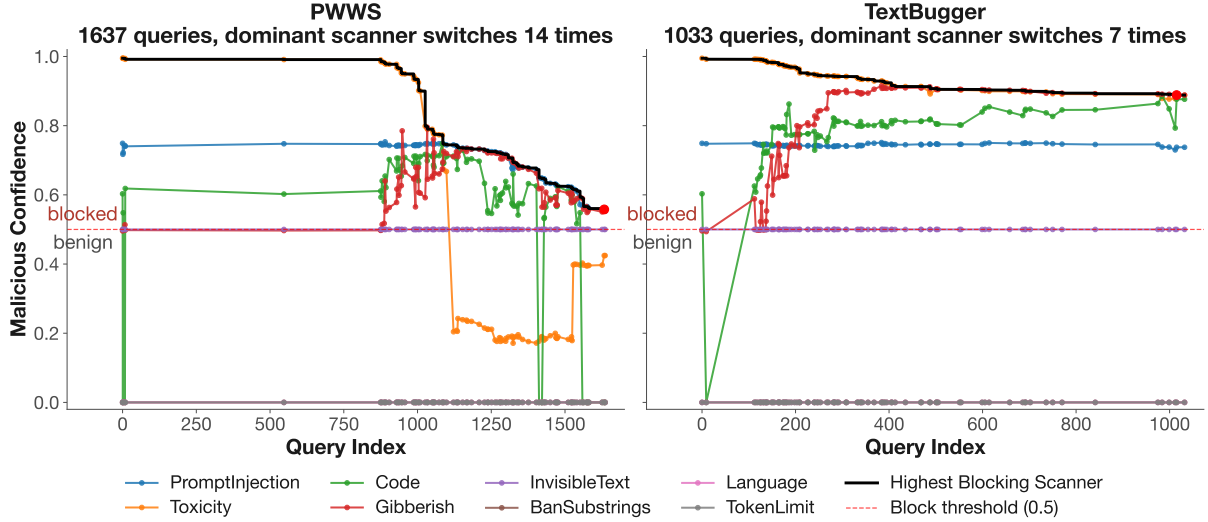


Figure 2: **Scanner Displacement.** Scanner confidences tracked across two single-objective techniques, PWWS and TextBugger showing the currently highest blocking scanner and variance of other scanners during perturbation.

their correlated feature spaces can act in opposition (Demontis et al., 2019).

Shared Latent Representations. If we consider a guardrail system comprising two scanners – prompt injection and jailbreak detection – both scanners may contain overlapping decision boundaries due to similar features typically associated with both prompt injections and jailbreaks (i.e. shared phrase and sentence structures that attempt to manipulate the model, or specific malicious payloads) (Rossi et al., 2024; Geng et al., 2026). An example of this overlap exists within AML literature, where prompt injection and jailbreaks are often conflated or misinterpreted (Rossi et al., 2024). Furthermore, prompt injection and jailbreaks are often embedded with other malicious content, such as toxicity or code, which can increase such overlap further (Li et al., 2025).

Static Scanner Activation. To quantify this overlap, we construct a dataset of 3,191 prompt injection and jailbreak samples sourced from PI-Guard (Li et al., 2025), and record the classification score of every scanner contained within the guardrail system LLM Guard for each sample. Figure 1a shows the resulting activation distribution, where 32.28% of samples activate no scanner, 47.57% activate exactly one, and the remaining 20.15% activate two or more. Figure 1b demonstrates that 7 of the 8 scanners activate at least once across the dataset, with Code accounting for the highest number activations at 1,287, followed by PromptInjection at 1,100. This spread indicates that despite every sample being labelled

as a prompt injection or jailbreak, the samples carry features that scanners trained for other categories detect. While only 20.15% of samples activate two or more scanners simultaneously, it is worth noting that these are static prompts measured before any perturbation is applied. Therefore, when an attacker applies perturbation to an input, it is likely that another scanner’s confidence score increases above the designated threshold and activates.

3.2 Scanner Interference

When a guardrail system contains multiple scanners, each query produces a separate confidence score per scanner rather than a single value. Current evasion techniques, such as PWWS and TextBugger, are designed around a single-objective, meaning they require a single confidence signal to guide the bypass search (Morris et al., 2020). Applying these techniques to a guardrail system therefore requires an aggregation function that collapses the per-scanner confidences into a singular scalar, all of which introduce their own trade-offs (Miettinen, 1999). However, collapsing multiple scanner confidence scores into a single scalar results in *scanner interference*, which from the perspective of the adversarial technique, this results in behavior of a scanner is indistinguishable from another, limiting the ability for a bypassing technique to optimize adversarial samples based upon feedback of all scanners.

Scanner Displacement. We demonstrate the problem of scanner interference by tracing every classification query made by two bypassing tech-

niques, PWWS and TextBugger, when applied to one seed prompt that performs adversarial perturbation in response to classification scoring against LLM Guard. Figure 2 shows that both techniques repeatedly suppress the currently dominating scanner only for a different scanner, with the dominant scanner switching 14 times over PWWS’s 1,637 queries and 7 times over TextBugger’s 1,033 queries leading the techniques to optimize against only the dominant scanner in those scenarios. This requires the techniques to navigate a more sophisticated decision boundary, learning to generate perturbations which appear to be effective against the target until another scanner becomes the dominant blocking scanner. Furthermore, these techniques typically perform word importance ranking before entering the perturbation stage, meaning the computed importance ranking is bespoke to the initial dominant scanner impacting effectiveness of the perturbations, and not representative of confidence across the entire guardrail system (Ren et al., 2019; Li et al., 2019). This interference therefore influences the effectiveness of these techniques, limiting their ability to generate bypasses when facing a multi-objective bypass scenario.

4 BRANCH

Addressing the challenges of single-objective bypassing techniques discussed in Section 3, we propose *BRANCH*, a multi-objective bypassing methodology that uses a branching tree search to bypass multiple scanners within a guardrail system. Each node in the search tree isolates a single *target scanner* that represents the blocking scanner with the highest confidence, and focuses perturbation techniques to target the scanner irrespective of changes to other scanner confidence scores, giving every perturbation a well-defined single objective. Once a perturbation technique completes, the resulting adversarial sample is considered a *candidate*, which are scored on the *total blocked confidence* summed across every scanner. This enables *BRANCH* to address scanner interference and overlapping decision boundaries by reducing the multi-objective bypass scenario into a single-objective, targeting only one individual scanner at a time, while rewarding perturbations that gradually bypass all scanners within the target. Therefore, by expanding the node with the lowest total blocked confidence and seeding each technique with the output of the last, *BRANCH* accumulates perturba-

tions that lower confidence across the guardrail system as a whole rather than attempting to suppress all scanners within one perturbation technique.

4.1 Threat Model

We consider an adversary whose goal is to generate a *bypass* – a prompt or instruction that successfully evades all scanners contained within a target guardrail system simultaneously, while preserving the malicious intent of the original seed prompt.

Access. We assume a *black-box* access model whereby the adversary can submit arbitrary prompts to the target guardrail system and observe the per-scanner blocking decision and associated confidence score for each query, yet has no access to model weights, gradients, internal architectures, or training data. This access model is representative of established bypassing literature where an attacker can access confidence scores by locally running the guardrail system, or through guardrail vendor APIs (Hackett et al., 2025; Li et al., 2019; Jin et al., 2020; Zhou et al., 2026).

Knowledge. The adversary requires knowledge of the blocking decision and confidence scores returned by a specific scanner and overall guardrail system. Knowledge obtained via local access to the guardrail system, or some cases moderation APIs (Hackett et al., 2025; Markov et al., 2023b). The adversary does not need any knowledge about the AI model, the training data, or AI system the guardrail system is operating within.

Capability. We assume the adversary is not constrained by a budget or rate limited. Our approach can be leveraged against open-source or self-hosted guardrails without vendor visibility, with successful bypasses generated subsequently applied to production guardrail systems (including unseen types). We report two metrics, *query* and *wall-clock* cost as measures of attack efficiency.

4.2 Preliminaries

Notation. Let G be our target guardrail system comprising k scanners, $G = \{s_1, \dots, s_k\}$, and P the original seed prompt considered for perturbation. Each scanner $s \in G$ produces a binary blocking decision b , along with a confidence score c , for an input prompt p .

$$b_s(p) \in \{0, 1\}, \quad c_s(p) \in [0, 1] \quad (1)$$

where $b_s(p) = 1$ indicates that scanner s blocks prompt p , and $c_s(p)$ is the associated with classification confidence. A prompt p is *blocked* by

the guardrail system if one or more scanner block due to exceeding the configured scanner confidence score threshold, and *evaded* if no scanner blocks it.

$$\begin{aligned} \text{blocked}(p) &= [\forall s \in G : b_s(p) = 1] \\ \text{evaded}(p) &= [\forall s \in G : b_s(p) = 0] \end{aligned} \quad (2)$$

Total Blocked Confidence. BRANCH tracks evasion progress against a guardrail system by measuring the total summed confidence scores across blocking scanners.

$$C(p) = \sum_{s \in G} b_s(p) \cdot c_s(p) \quad (3)$$

where $C(p)$ is the total summed confidence of currently blocking scanners in the guardrail system.

Target Scanner. The target scanner for p is the blocking scanner returning the highest confidence.

$$\sigma^*(p) = \arg \max_{s \in G : b_s(p)=1} c_s(p) \quad (4)$$

where $\sigma^*(p)$ represents the current target scanner with the highest confidence across all blocking scanners within the guardrail system.

Tree Structure. BRANCH maintains a search tree $T = (V, E)$ beginning with P where each node $v \in V$ holds a prompt p_v , and each edge in E records the node a candidate originated from. Each node contains its own target scanner, $\sigma^*(p_v)$, total blocked confidence $C(p_v)$, and depth d_v with two optional constraints: $D_{\max}$ for maximum depth, and $N_{\max}$ for maximum nodes expanded.

Bypass Candidate Set. BRANCH runs an ensemble of evasion techniques to generate potential bypasses which are leveraged as a set of candidates $\mathcal{C}_v = \{q_1, \dots, q_m\}$, where each q_i is a modified variant of p_v generated against the target scanner.

4.3 Methodology

BRANCH shown in Algorithm 1, addresses the challenges that established techniques have against guardrail systems by focusing each node’s search to a single scanner, rather than optimizing all scanners simultaneously. At each step, BRANCH selects the pending node with the lowest total blocked confidence across all scanners, prioritizing the prompt closest to evasion the the scanner returning the lowest confidence score, and then processes it in three phases. We refer to a branch which leads to a successful bypass as an *optimal path*. We outline an example of this process in Figure 3.

Algorithm 1: BRANCH algorithm

Input: initial seed prompt P ; guardrail system G ; perturbation techniques $\mathcal{T}$; node budget $N_{\max}$; max depth $D_{\max}$

Output: set of evaded prompts P^*

```

1  $P^* \leftarrow \emptyset$ 
2 evaluate  $P$  against  $G$  // gather baseline
3  $V_{\text{pending}} \leftarrow \{\text{make\_root}(P) \mid P \text{ blocked}\}$ 
4  $n \leftarrow 0$ 
5 while  $V_{\text{pending}} \neq \emptyset$  and  $n < N_{\max}$  do
6    $v^* \leftarrow \arg \min_{v \in V_{\text{pending}}} C(p_v)$  // select node
   based on  $C(p_v)$ 
7    $V_{\text{pending}} \leftarrow V_{\text{pending}} \setminus \{v^*\}$ 
8    $n \leftarrow n + 1$ 
9   /* Phase 1: Bypass Target Scanner */
    $\mathcal{C}_{v^*} \leftarrow \text{BYPASS}(p_{v^*}, \sigma^*(p_{v^*}), \mathcal{T})$  // bypass
   job
10  /* Phase 2: Validate */
   forall  $q_i \in \mathcal{C}_{v^*}$  do
11    evaluate  $q_i$  against guardrail system  $G$ 
12    if  $\text{evaded}(q_i)$  then
13       $P^* \leftarrow P^* \cup \{q_i\}$ 
14  if  $P^* \neq \emptyset$  then
15    return  $P^*$  // early exit on bypass
16  /* Phase 3: Grow */
   if  $d_{v^*} < D_{\max}$  then
17    forall blocked  $q_i \in \mathcal{C}_{v^*}$  and not in tree do
18       $v' \leftarrow (q_i, d_{v^*} + 1)$ 
19       $V_{\text{pending}} \leftarrow V_{\text{pending}} \cup \{v'\};$ 
       $E \leftarrow E \cup \{(v^*, v')\}$ 
20 return  $P^*$ 
```

Phase 1: Bypass. Generates prompt candidates modified through adversarial perturbation to degrade the performance of, or fully bypass, a specific target scanner. Perturbations are handled by an ensemble of techniques, which we term a *bypass job*. As BRANCH is agnostic to the evasion technique, any collection of perturbation methods can be used. Furthermore, isolating to a single scanner enables single-objective perturbation techniques to operate within their limitations.

Phase 2: Validate. Focuses on validating generated candidates against the guardrail system. This phase ensures BRANCH considers the multi-objective nature of bypassing every scanner in a guardrail system. A candidate which triggers the guardrail system is recorded as blocked, whereas any candidates that are not blocked are reported as bypasses that will result in the search subsequently terminating.

Phase 3: Grow. Responsible for expanding the search tree, every candidate prompt produced which does not already exist within the tree, and is

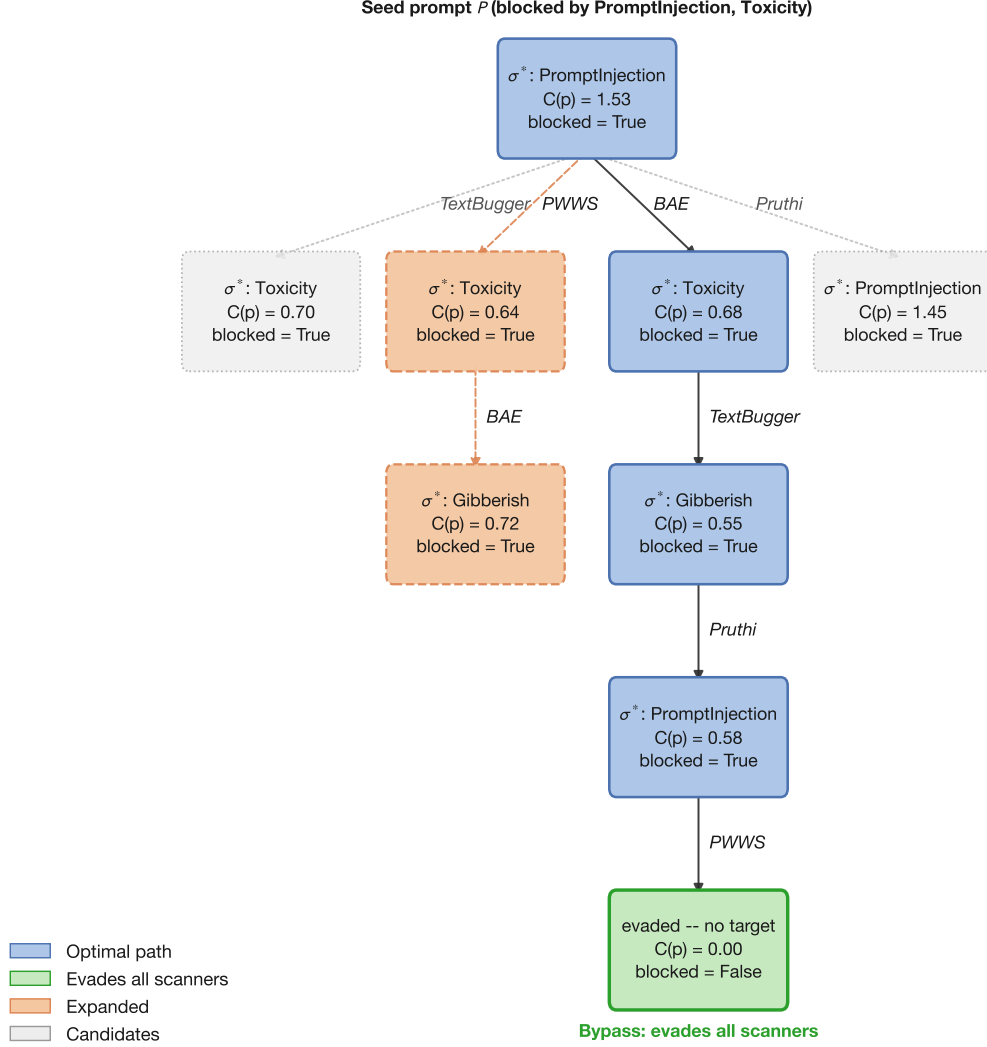


Figure 3: **BRANCH Methodology.** Example bypass against a guardrail system with three scanners: Prompt Injection, Toxicity, and Gibberish, using four perturbation techniques BAE, TextBugger, Pruthi, and PWWS. BRANCH selects the node with the lowest total blocked confidence ($C(p)$), perturbs it with one or more techniques, and then validates candidates against the full guardrail system. The process is repeated until a bypass is found.

blocked by the guardrail system becomes a child node at that specific depth. Each node contains the candidate prompt it should perturb, and the target scanner which currently has the highest confidence with that candidate. Candidates are then utilized as the input to the next node within the tree, enabling techniques to be chained together, and therefore exploring a more sophisticated search space.

5 Experimental Setup

Evaluation approaches. We compare BRANCH against 11 prominent guardrail bypassing techniques (see Appendix Table A.1), selected based upon their diversity in perturbation methodologies (Morris et al., 2020). We evaluate three approaches.

Individual Techniques: Each individual technique applies adversarial perturbation to a seed prompt to measure the effectiveness of a single isolated bypass. *Exhaustive:* All 11 techniques are ran, whereby a successful bypass is determined in the event of *any* technique succeeding. *BRANCH:* Our approach, which chains techniques across the search tree so that the output of one seeds the next. The query budgets differ across approaches, where individual techniques and Exhaustive utilize an unbounded query budget, terminating only when the technique exhausts its search, while BRANCH uses a maximum of 5,000 queries per node, a node budget $N_{\max} = 120$, and maximum depth $D_{\max} = 120$, selected to evaluate BRANCH’s ef-

Table 1: **Target Guardrail Systems.** Guardrails targeted in this work, including total number of scanners.

Guardrail System	Number of Scanners
LLM Guard (Protect AI, 2025)	8
Vijil Dome (Vijil AI, 2024)	3
Guardrails AI (Guardrails AI, 2026)	3
Samsung Guard (Lee et al., 2025)	2
LlamaFirewall (Chennabasappa et al., 2025)	2
Ensemble Guardrail (Custom Built)	3

fectiveness and efficiency across both depth and breadth of the search tree.

Targets. We evaluate approaches against 6 guardrail systems listed in Table 1, each comprising multiple scanners across detection categories (see Appendix Table A.2). We use each guardrail system default confidence score threshold and, where no default is provided, set the threshold to 0.50. Five of the six evaluated guardrail systems are established open-source or commercial guardrail systems, while the sixth, Ensemble Guardrail, is assembled from three popular open-source HuggingFace models covering distinct detection categories: prompt injection and jailbreak (Rogue Security, 2025), toxicity (Hanu and Unitary team, 2020), and gibberish (Jindal, 2021), to evaluate a custom-built guardrail system constructed of open-source models varying in architecture, training, and task.

Bypass Candidates. Candidates are selected from the PIGuard dataset, which are filtered to contain only malicious prompts across prompt injection and jailbreaks (Li et al., 2025). We conducted an initial experiment to rank and select the top 20 prompts that were blocked by the highest number of scanners across our target guardrail systems, so the approaches are evaluated against the strictest bypass scenario. This results in 120 *bypassing scenarios* across 6 guardrail systems, with 20 seed prompts per system, respectively.

System Setup. Experiments were conducted on a Linux Ubuntu 20.04 server with 128GB RAM, an Intel Xeon Gold 6336Y CPU, and two A100 GPU (80GB VRAM). Experiments scenarios were ran serially to minimize performance interference.

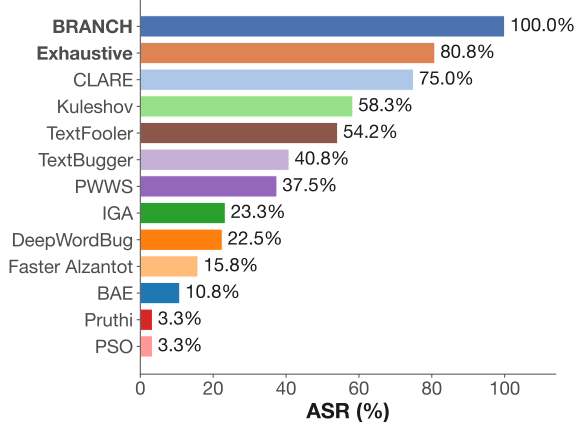
Evaluation Metrics. We evaluate our approach using three metrics. *Attack Success Rate (ASR)*: The percentage of prompts that successfully evade all scanners in a guardrail system, *Query Efficiency*: The average number of queries required to generate a successful bypass, and *Wall-Clock Time*: The average time taken to generate a successful bypass.

Bypass Similarity. To evaluate how well the generated bypasses preserve the original seed prompts adversarial goal, we leverage two metrics: *Surface Similarity*: A character-level sequence ratio between the original and bypass text, and *Semantic Preservation*: The cosine similarity between sentence embeddings of the original and bypass text (Reimers and Gurevych, 2019). Furthermore, to measure the *adversarial utility* of the generated bypasses, we utilize an ablated Qwen 3 8B model selected as a control to observe whether the perturbation introduces divergence in model behaviour rather than the effectiveness of the bypass sample to overcome the model’s safety alignment (Young, 2026). To measure the utility, we use an LLM Judge, utilising gpt-5.6-terra to compare the ablated model’s generated output when provided the original seed prompt and the generated bypass, scoring similarity from 1 to 5 (Zheng et al., 2024).

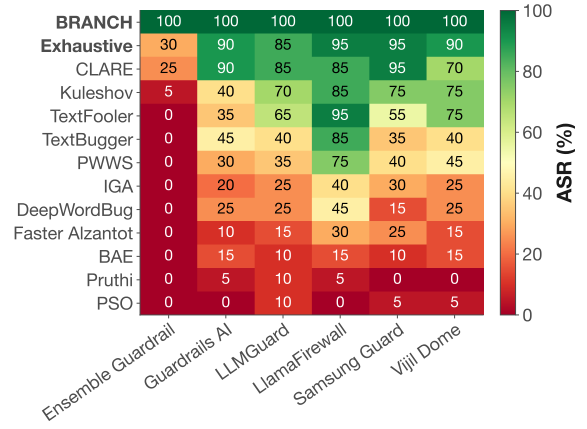
6 Results

6.1 Bypass Effectiveness

Attack Success Rate. Figure 4a shows the overall guardrail system bypass ASR across all approaches. BRANCH successfully generated a bypass in all 120 scenarios achieving 100% ASR, compared to 80.8% for Exhaustive. Focusing on individual techniques, CLARE was the most effective with an ASR of 75.0%, followed by Kuleshov at 58.3% and TextFooler at 54.2%, while the weakest Pruthi and PSO attain 3.3% each. Exhaustive reaches 80.8%, 5.8% higher than CLARE, indicating that the remaining 10 techniques therefore yielded limited additional coverage. Figure 4b shows BRANCH reaches 100% success across all 6 guardrail systems, while Exhaustive techniques average between 85% and 95% on five of these targets, with Ensemble Guardrail being the most robust against these methods at 30%. We attribute the difficulty in bypassing Ensemble Guardrail due to a higher number of scanner activations compared to other targets (see Appendix Figure A.1), where on average more than 1 scanner is activated initially leading to more overlapping boundaries. We observe that BRANCH’s gains are additive, as across all bypass scenarios there is no singular bypass scenario whereby an Exhaustive technique succeeds that BRANCH fails against, bypassing 23 additional scenarios than Exhaustive, with CLARE accounting for the large majority of the coverage (see Appendix Figure A.3).

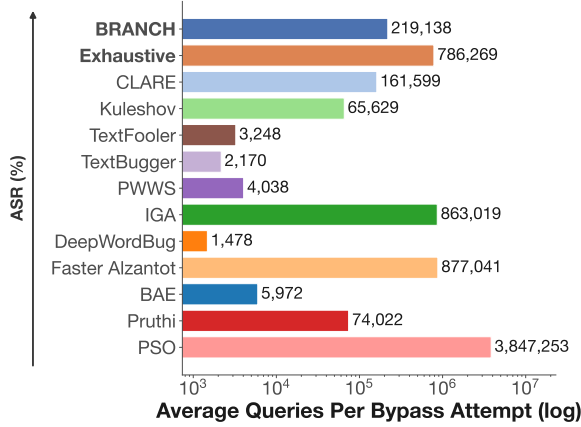


(a) Bypass effectiveness.

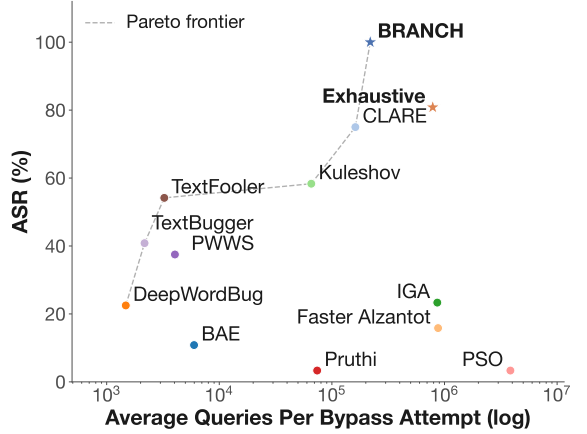


(b) Heat map by guardrail target.

Figure 4: **Bypass Effectiveness.** Overall attack success rate of all methods against 6 guardrail systems.



(a) Query Efficiency.



(b) ASR vs Query efficiency.

Figure 5: **Query Efficiency Across Methods.** Average queries per bypass attempt for BRANCH, Exhaustive, and the 11 individual techniques, across targeted guardrail system.

Efficiency. Figure 5a shows that BRANCH spends 219,138 queries on average per bypass compared to 786,269 for Exhaustive, a 72.13% reduction, or 3.58x fewer queries. Exhaustive’s higher bypass cost is driven by scenarios it fails to bypass, running to exhaustion under an unbounded budget without ever finding a successful bypass. Even assuming identical budget between BRANCH and Exhaustive, this results in the average ASR of the latter dropping from 80.8% to 75.0% utilizing the most effective individual technique CLARE, and to 53.9% if techniques are selected at random. Several individual techniques require fewer queries per bypass than BRANCH, including TextFooler and DeepWordBug, however attain substantially lower ASR by 45.8-89.2 percentage points. Figure 5b places every method on the ASR vs query-efficiency plane, where BRANCH lies on the top

of the Pareto frontier. We observe that query cost is highest when targeting the Ensemble Guardrail across all bypassing techniques. Wall-clock time follows the same pattern, where BRANCH completes a bypass job in 2.9h compared to 13.1h for Exhaustive (4.51x speed-up). Among the individual techniques, the closest in ASR, CLARE, is slower at 5.8h, while faster techniques achieve lower ASRs (see Appendix Figure A.2). While individual techniques such as CLARE can achieve high ASR, they do so only in specific bypass scenarios, meaning an adversary would be required to run multiple techniques in case one fails to achieve a bypass, therefore decreasing bypass efficiency. BRANCH is more efficient than Exhaustive, relaxing the requirement for an adversary to know which technique is best given a bypass scenario, achieving a higher ASR while consuming fewer

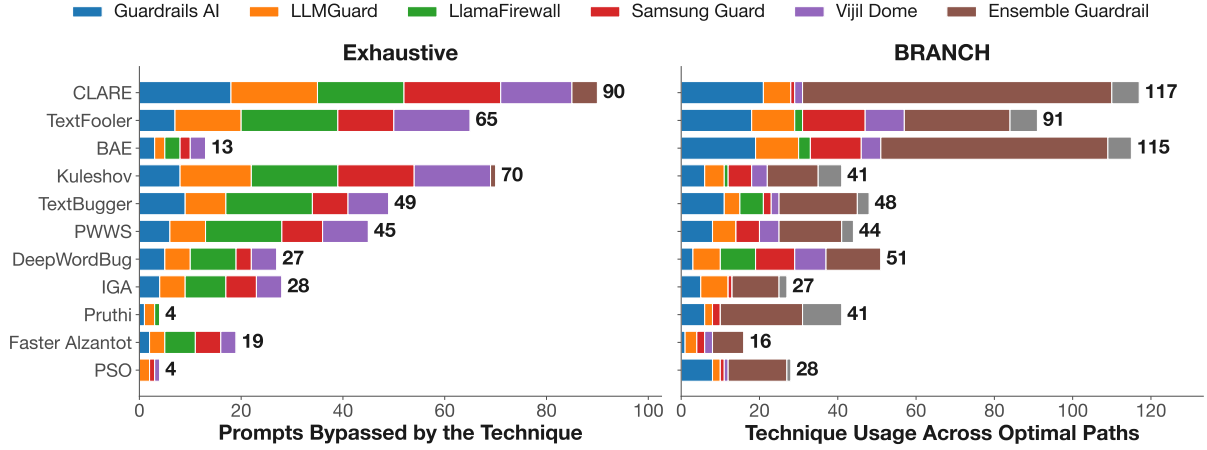


Figure 6: **Most Successful Perturbation Techniques.** The left panel shows the number of prompts bypassed by each Exhaustive technique alone. The right panel shows the number of times each technique is used along BRANCH’s optimal paths.

queries and less wall-clock time.

BRANCH Technique Composition. We observe in Figure 6 that BRANCH enabled less prominent bypassing techniques from Exhaustive to contribute meaningfully once chained together in BRANCH. For example, BAE and Pruthi-generated bypasses increase from 13 to 115 and 4 to 41, respectively within BRANCH’s designation of the optimal bypass paths. In contrast, dominant techniques such as Kuleshov are used comparatively less (70 in Exhaustive, 41 in BRANCH). This indicates that techniques, especially those with lower ASR, contribute meaningfully to the final bypass within BRANCH, where when applied in isolation they struggle to bypass the target. We attribute this phenomenon to the specificity of bypass techniques. Where individual approaches typically utilize unique search and perturbation methodologies designed to exploit specific weaknesses within classification models. Chaining these techniques therefore enables them to share optimizations, creating bypasses which combine the advantages of multiple approaches.

6.2 Generated Bypass Similarity

As stated in Section 5, beyond generating bypasses, it is important to determine whether the successful bypass still contains the same lexical meaning. We measured the original seed prompt to the generated bypass sample using two metrics: *surface similarity* and *semantic preservation*. We also measure the *adversarial utility* of the bypass, observing if an ablated LLM interprets the bypass in the same way compared with the original seed prompt.

Surface Similarity and Semantic Preservation.

Figure 7 depicts the ASR and average semantic preservation of each approach. All 13 methods fall within a similar preservation deviation when evaluated across all targets. BRANCH achieves comparable average semantic preservation to Exhaustive across targets, such as 0.86 vs 0.87 for LLM Guard, 0.81 vs 0.82 for LlamaFirewall, 0.83 vs 0.85 for Vijil Dome, and 0.72 vs 0.76 for Ensemble, with the biggest difference against Guardrails AI at 0.65 vs 0.81 (Appendix Table A.3). Surface similarity is where the two approaches greatly diverge, despite BRANCH preserving semantic content, the produced bypasses are modified more compared to Exhaustive and individual techniques, with BRANCH having an average surface similarity of 0.60 compared to 0.71 for Exhaustive (Appendix Figure A.4). We attribute this to the side-effect of chaining perturbations together where the specific techniques such as IGA, and TextFooler appear to apply aggressively more modifications which can reduce surface similarity. Despite BRANCH applying more surface-level perturbations, the resulting bypass is still semantically similar to the seed prompt, and additionally comparable to established approaches. Therefore, not only does BRANCH achieve higher ASR compared to other bypass approaches, it does so while retaining the underlying meaning of the original prompt.

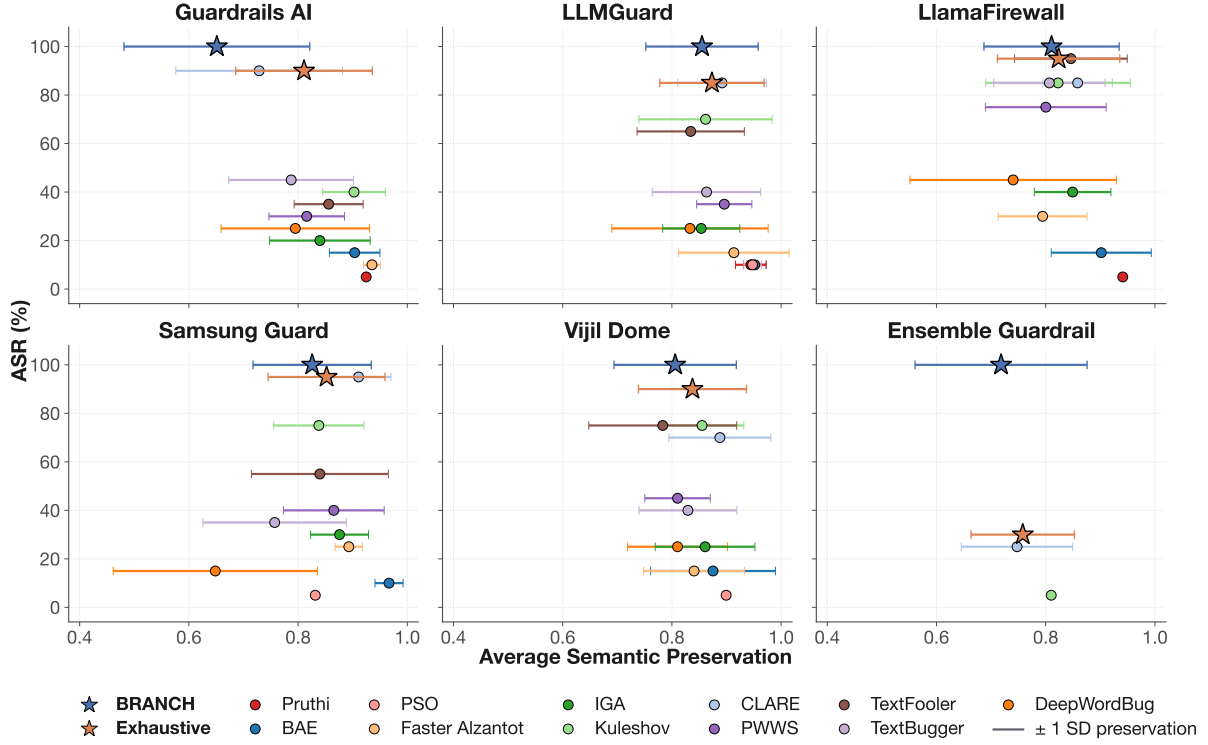


Figure 7: **Semantic Preservation.** Average semantic preservation of each technique’s generated bypasses plotted against ASR. The stars mark BRANCH and Exhaustive, while dots represent individual techniques. Horizontal bars span ± 1 standard deviation of semantic preservation across all generated bypasses.

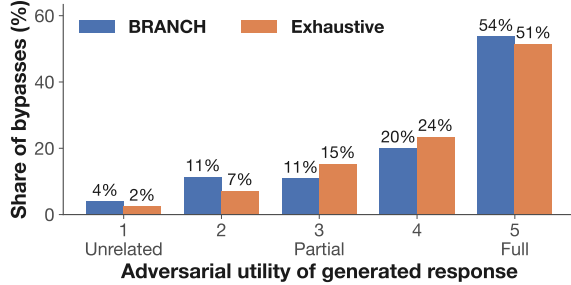


Figure 8: **Adversarial Utility.** Distribution of adversarial utility scores comparing the outputs generated from the original seed prompt and generated bypasses, evaluated by an LLM judge. Utility is scored from 1-5 (1 indicates no similarity, 5 indicates high similarity).

BRANCH Depth Trade-off. We further analyzed how surface similarity and semantic preservation decays as a function of tree depth (Appendix Figures A.5 and A.6). Surface similarity decays sharply and continues to fall as depth increases across all 6 targets, from 1.0 to 0.23 by depth 21 on Guardrails AI, and to 0.14 by depth 13 on Samsung Guard. Semantic preservation decays far more gradually over the same depth range, typically plateauing after the first few layers, remaining between 0.85 and 0.88 across depths 2-10 on

LLM Guard, before lowering to 0.72-0.77 in further depths. This gap between the two curves increases with depth, indicating that BRANCH accumulates larger surface-level perturbations as the depth of the tree grows while largely preserving the semantic content of the original request. As the semantic meaning is comparable to established approaches, this highlights that BRANCH is capable of exploring more sophisticated perturbation spaces compared to other techniques, modifying the underlying prompt more to achieve a bypass.

Adversarial Utility. We evaluated the adversarial utility of the generated bypasses against an ablated Qwen 3 8B model to determine whether the bypasses generate outputs which are similar to those from the original seed prompt. Figure 8 shows that BRANCH generates comparable utility to the Exhaustive, with 85% of BRANCH bypasses scoring at or above 3 compared to 90% for Exhaustive, with 54% of bypasses scoring 5 (indicating very high similarity) compared to 51% for Exhaustive. These results highlight that the generated bypasses generate outputs against the ablated model which are similar to those generated using original seed prompts, indicating that intent is preserved through the bypassing process.

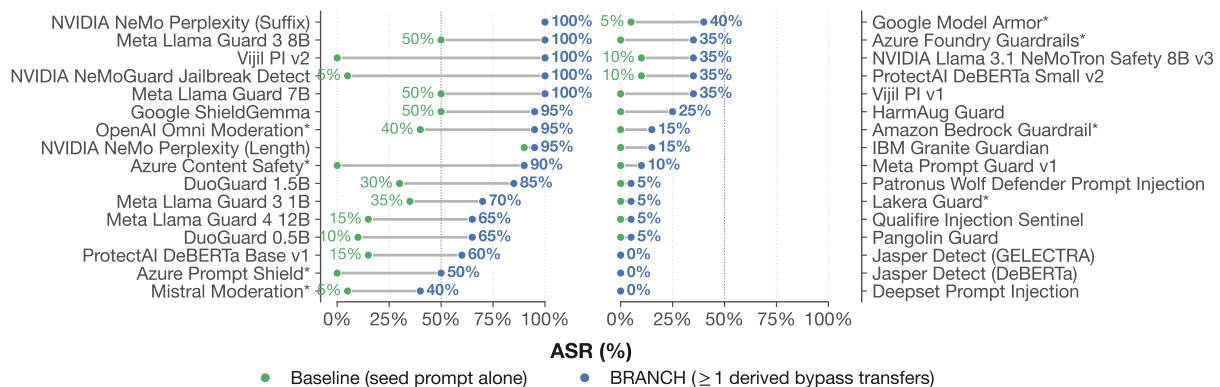


Figure 9: **Transferability to Unseen Guardrails.** Per-guardrail ASR of the 20 original seed prompts alone as a baseline against that of BRANCH’s generated bypasses, for each of the 32 previously-unseen guardrails, ordered by transfer ASR. A seed prompt counts as transferring if at least one bypass derived from a given seed evades the guardrail. * = black-box commercial guardrails.

6.3 Transferability

To assess transferability of bypasses generated by BRANCH, we collect every successful bypass generated by BRANCH and evaluate them against a suite of 32 unseen white-box and black-box guardrail scanners, systems, and moderation LLMs. We measure transferability by assessing the number of bypasses derived from a seed prompt that evades an unseen guardrail.

Transferability to Unseen Scanners. As shown in Figure 9, we observe that BRANCH bypasses successfully transfers to 29 out of 32 unseen guardrails never queried during initial generation. We observe that the ASR per scanner ranges from 5% on the most resistant scanners to 100% on 5 scanners that are evaded on every seed prompt include (Meta Llama Guard 3 8B, NVIDIA NeMoGuard Jailbreak Detect, Vijil PI v2). Moreover, we note that generated bypasses transferring to unseen scanners appear to be exploiting shared weaknesses where specific targets are more likely to transfer to one another, rather than a generalized weakness across all scanners. For example, Guardrails AI appears to be the best target to generate bypasses that transfer to Meta Llama Guard 3 1B, and Meta Llama Guard 3 8B with 70% and 77% ASR, respectively, while Vijil Dome was the best target for Azure Foundry Guardrails and NVIDIA Llama 3.1 NeMoTron Safety Guard 8B v3 (see Appendix Figure A.7 for transfer ASR by target).

Guardrail Transfer Effectiveness. Utilizing BRANCH-generated prompts, ASR increased on average of 32.9 percentage points. We observe the largest ASR increases were NVIDIA NeMoGuard Jailbreak Detect and Vijil PI v2 (90%), and Azure

Content Safety (90%). Scanners with moderate baselines behave similarly, including DuoGuard 0.5B (55%) and OpenAI Omni Moderation from (55%). 8 evaluated guardrails are commercial black-box offerings (no access to confidence scores) that exhibit an ASR increase within Lakera Guard (5%), Amazon Bedrock Guardrail (15%), Azure Foundry Guardrails (35%), Google Model Armor and Mistral Moderation (35%), Azure Prompt Shield (50%), Azure Content Safety (90%) and OpenAI Omni Moderation (55%).

These findings demonstrate that an attacker is capable of generating bypasses against a surrogate guardrail system and at no additional cost, transferring the bypasses to other unseen guardrails, including commercial black-box offerings.

7 Limitations

Candidate Prompt Selection. Our evaluation uses 20 seed prompts per guardrail system, totalling 120 bypass scenarios. These prompts were selected as those blocked by the most scanners across our targets (see Appendix Figure A.1), ensuring experimentation was conducted within a strict multi-objective scenario. In doing so, full evaluation across the 120 bypass scenarios took a total of 1,923.8 hours, approximately 80 days. This cost was driven by the unbounded search used by Exhaustive and the individual techniques, which limited the number of bypass scenarios we could evaluate in favor or comparing to established work. While we believe these prompts sufficiently demonstrate the challenges faced by established techniques when targeting guardrail systems, and that BRANCH can address these limitations, fu-

ture work can extend BRANCH to explore other candidate prompts sourced from other datasets.

Hyperparameter Optimization. In our experiments, BRANCH uses a query budget of 5,000 per node greatly reduced from the unbounded budget utilized in Exhaustive. We believe there is opportunity to refine this further through optimization of the utilized query budget based upon parent nodes within the tree and the selected perturbation technique. Doing so could enable BRANCH to be more efficient given the context of the target and historical branches in the tree, enabling BRANCH to achieve bypasses earlier with fewer queries per node and terminating at a shallower depth.

Threat Model. BRANCH assumes an adversary can observe the per-scanner confidence scores and blocking decisions of the target guardrail system. This assumption is realistic when an adversary can access the guardrail system locally or if vendor APIs expose per-scanner outputs, typically offered by commercial guardrails to enable developers to configure detection thresholds, however would be less effective in strict black-box production scenarios where confidence scores and guardrail activation is hidden (Hackett and Garraghan, 2026). While this limitation is focused upon the generation of the bypass samples, our transferability study highlights that even in a strict black-box threat models, an adversary can generate bypasses offline which successfully transfer to an unseen target without requiring to perform optimization (Section 6.3).

Bypass Efficacy. Our study focuses primarily on the task of bypassing guardrails. While we interoperate various metrics to measure adversarial utility, such as semantic preservation, surface similarity, and utilizing an ablated model to observe if the intent of the bypass prompt matches that of the original, we do not conduct a thorough experiment on the efficacy of the bypassed prompt against production LLMs. Therefore, we believe there exists further work to assess the utility of the bypassed prompts and the effect BRANCH and various techniques have on the overall ASRs of the prompts.

8 Related Work

Adversarial ML evasion techniques have been used to successfully bypass text-classification models (Pruthi et al., 2019; Li et al., 2019; Gao et al., 2018; Kuleshov et al., 2018; Wang et al., 2021) and more recently, guardrails. However, their respective evaluation focus isolated single-scanner

guardrails rather than multi-scanner guardrail systems (Hackett et al., 2025; Wang et al., 2026). For example, early techniques such as TextFooler (Jin et al., 2020), BERT-Attack (Li et al., 2020), and PWWS (Ren et al., 2019) demonstrated their effectiveness against sentiment analysis and classification models, which were later applied to widely used classification models in guardrails (Hackett et al., 2025). More recent guardrail bypassing approaches exploit different weaknesses, such as hiding content outside the input window of the guardrail (Zhou et al., 2026), or exploiting representation asymmetry between guardrail classifiers and the LLM, where minor character perturbations that confuse the guardrail remain interpretable to the target model (Wang et al., 2026). All of these studies are evaluated against multiple single-scanner guardrails, and do not consider scenarios whereby multiple scanners are deployed within a guardrail system that require bypasses to evade all scanners in order to succeed.

9 Conclusion

In this paper we have proposed BRANCH – a methodology designed to bypass multi-scanner guardrail systems. Our work identifies and targets the gap in current evasion attack and guardrail research where existing techniques struggle to converge on adversarial prompts capable of bypassing guardrail systems containing multiple scanners. Across 6 target guardrail systems and 120 bypass scenarios, BRANCH is capable of attaining 100% bypass success rate while consuming 72% fewer queries and 4.5x less wall-clock time per bypass. Importantly, we demonstrate that generated bypasses transfer reasonably well to a large number of unseen guardrail systems, including commercial offerings. We hope our findings will help the research community to create more advanced evasion attack techniques applicable to more sophisticated AI system defenses, as well as help practitioners evaluate their guardrail system effectiveness.

10 Acknowledgments

We would like to thank members of the Lancaster University AI Security Lab and Mindgard research team for their comments and feedback. This work was supported by the UK Engineering Physical Science and Research Council (EPSRC) and Mindgard.

References

- Moustafa Alzantot, Yash Sharma, Ahmed Elgohary, Bo-Jhang Ho, Mani Srivastava, and Kai-Wei Chang. 2018. [Generating natural language adversarial examples](#). *Preprint*, arXiv:1804.07998.
- Suriya Ganesh Ayyamperumal and Limin Ge. 2024. Current state of llm risks and ai guardrails. *arXiv preprint arXiv:2406.12934*.
- Elias Bassani, Ignacio Sanchez, and 1 others. 2025. On guardrail models’ robustness to mutations and adversarial attacks. *Findings of the Association for Computational Linguistics: EMNLP*, 2025.
- Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Šrđić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. 2013. Evasion attacks against machine learning at test time. In *Machine Learning and Knowledge Discovery in Databases*, pages 387–402, Berlin, Heidelberg. Springer Berlin Heidelberg.
- Steven Bird and Edward Loper. 2004. [NLTK: The natural language toolkit](#). In *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, pages 214–217, Barcelona, Spain. Association for Computational Linguistics.
- Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. 2025. [Llamafirewall: An open source guardrail system for building secure ai agents](#). *Preprint*, arXiv:2505.03574.
- Ambra Demontis, Marco Melis, Maura Pintor, Matthew Jagielski, Battista Biggio, Alina Oprea, Cristina Nita-Rotaru, and Fabio Roli. 2019. [Why do adversarial attacks transfer? explaining transferability of evasion and poisoning attacks](#). *Preprint*, arXiv:1809.02861.
- Yi Dong, Ronghui Mu, Yanghao Zhang, Siqi Sun, Tianle Zhang, Changshun Wu, Gaojie Jin, Yi Qi, Jinwei Hu, Jie Meng, Saddek Bensalem, and Xiaowei Huang. 2024. [Safeguarding large language models: A survey](#). *Preprint*, arXiv:2406.02622.
- Abhimanyu Dubey and et al. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Ji Gao, Jack Lanchantin, Mary Lou Soffa, and Yanjun Qi. 2018. [Black-box generation of adversarial text sequences to evade deep learning classifiers](#). *Preprint*, arXiv:1801.04354.
- Siddhant Garg and Goutham Ramakrishnan. 2020. [BAE: BERT-based adversarial examples for text classification](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6174–6181, Online. Association for Computational Linguistics.
- Tongcheng Geng, Zhiyuan Xu, Yubin Qu, and W. Eric Wong. 2026. [Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies](#). *Computers, Materials and Continua*, 87(1).
- Guardrails AI. 2026. [Guardrails ai](#). <https://guardrailsai.com/>. Accessed: 2026-07-04.
- William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. 2025. [Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems](#). In *Proceedings of the The First Workshop on LLM Security (LLMSEC)*, pages 101–114, Vienna, Austria. Association for Computational Linguistics.
- William Hackett and Peter Garraghan. 2026. [Behind the refusal: Determining guardrail activation via behavioral monitoring](#). *Preprint*, arXiv:2607.02121.
- Laura Hanu and Unitary team. 2020. [Unbiased toxic roberta](#).
- Robin Jia, Aditi Raghunathan, Kerem Göksel, and Percy Liang. 2019. [Certified robustness to adversarial word substitutions](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4129–4142, Hong Kong, China. Association for Computational Linguistics.
- Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. 2020. [Is bert really robust? a strong baseline for natural language attack on text classification and entailment](#). *Preprint*, arXiv:1907.11932.
- Madhur Jindal. 2021. [Autonlp gibberish detector](#).
- Volodymyr Kuleshov, Shantanu Thakoor, Tingfung Lau, and Stefano Ermon. 2018. [Adversarial examples for natural language classification problems](#).
- JoonHo Lee, HyeonMin Cho, Jaewoong Yun, Hyunjae Lee, JunKyu Lee, and Juree Seok. 2025. [Sguard-v1: Safety guardrail for large language models](#). *Preprint*, arXiv:2511.12497.
- Dianqi Li, Yizhe Zhang, Hao Peng, Liqun Chen, Chris Brockett, Ming-Ting Sun, and Bill Dolan. 2021. [Contextualized perturbation for textual adversarial attack](#). In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5053–5069, Online. Association for Computational Linguistics.
- Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao. 2025. [Piguard: Prompt injection guardrail via mitigating overdefense for free](#). *ACL*.
- Jinfeng Li, Shouling Ji, Tianyu Du, Bo Li, and Ting Wang. 2019. [Textbugger: Generating adversarial text against real-world applications](#). In *Proceedings 2019 Network and Distributed System Security Symposium, NDSS 2019*. Internet Society.

- Linyang Li, Ruotian Ma, Qipeng Guo, Xiangyang Xue, and Xipeng Qiu. 2020. [Bert-attack: Adversarial attack against bert using bert](#). *Preprint*, arXiv:2004.09984.
- Todor Markov, Chong Zhang, Sandhini Agarwal, Tyna Eloundou, Teddy Lee, Steven Adler, Angela Jiang, and Lilian Weng. 2023a. [A holistic approach to undesired content detection in the real world](#). *Preprint*, arXiv:2208.03274.
- Todor Markov, Chong Zhang, Sandhini Agarwal, Florentine Eloundou Nekoul, Theodore Lee, Steven Adler, Angela Jiang, and Lilian Weng. 2023b. A holistic approach to undesired content detection in the real world. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 15009–15018.
- Microsoft Corporation. 2024. [Content filtering in azure openai service](#). Accessed: 2024-08-06.
- Kaisa Miettinen. 1999. *Nonlinear Multiobjective Optimization*, volume 12 of *International Series in Operations Research & Management Science*. Springer, Boston, MA.
- John Morris, Eli Lifland, Jin Yong Yoo, Jake Grigsby, Di Jin, and Yanjun Qi. 2020. Textattack: A framework for adversarial attacks, data augmentation, and adversarial training in nlp. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 119–126.
- Aleksandra Nastoska, Bojana Jancheska, Maryan Riziniski, and Dimitar Trajanov. 2025. [Evaluating trustworthiness in ai: Risks, metrics, and applications across industries](#). *Electronics*, 14(13).
- Protect AI. 2025. [Llm guard: The security toolkit for llm interactions](#). <https://github.com/protectai/llm-guard>. Accessed: 2026-07-04.
- Danish Pruthi, Bhuwan Dhingra, and Zachary C. Lipton. 2019. [Combating adversarial misspellings with robust word recognition](#). *Preprint*, arXiv:1905.11268.
- Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. [Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails](#). *Preprint*, arXiv:2310.10501.
- Nils Reimers and Iryna Gurevych. 2019. [Sentence-BERT: Sentence embeddings using Siamese BERT-networks](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China. Association for Computational Linguistics.
- Shuhuai Ren, Yihe Deng, Kun He, and Wanxiang Che. 2019. [Generating natural language adversarial examples through probability weighted word saliency](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1085–1097, Florence, Italy. Association for Computational Linguistics.
- Rogue Security. 2025. [Prompt injection & jailbreak sentinel v2](#).
- Sippo Rossi, Alisia Marianne Michel, Raghava Rao Mukkamala, and Jason Bennett Thatcher. 2024. [An early categorization of prompt injection attacks on large language models](#). *Preprint*, arXiv:2402.00898.
- Aditya S Shethiya. 2023. Llm-powered architectures: Designing the next generation of intelligent software systems. *Academia Nexus Journal*, 2(1).
- Vijil AI. 2024. [Vijil dome](#). <https://github.com/vijilAI/vijil-dome>. Accessed: 2026-07-04.
- Junyi Wang, Zhibin Zhu, and Chuanyi Liu. 2026. [The asymmetric vulnerability: Bypassing llm defenses via guardrail-model mismatch](#). In *Proceedings of the ACM Web Conference 2026*, WWW '26, page 3078–3089, New York, NY, USA. Association for Computing Machinery.
- Xiaosen Wang, Hao Jin, Yichen Yang, and Kun He. 2021. [Natural language adversarial defense through synonym encoding](#). *Preprint*, arXiv:1909.06723.
- Fangzhou Wu, Ning Zhang, Somesh Jha, Patrick McDaniel, and Chaowei Xiao. 2024. A new era in llm security: Exploring security concerns in real-world llm-based systems. *arXiv preprint arXiv:2402.18649*.
- Richard J. Young. 2026. [Comparative analysis of llm abilitation methods: A cross-architecture evaluation](#). *Preprint*, arXiv:2512.13655.
- Yuan Zang, Fanchao Qi, Chenghao Yang, Zhiyuan Liu, Meng Zhang, Qun Liu, and Maosong Sun. 2020. [Word-level textual adversarial attacking as combinatorial optimization](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6066–6080, Online. Association for Computational Linguistics.
- Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. 2024. [Shield-gemma: Generative ai content moderation based on gemma](#). *Preprint*, arXiv:2407.21772.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2024. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA. Curran Associates Inc.

Yuanbo Zhou, Changjia Zhu, Junyu Wang, Xu He, Yan Zhai, Kun Sun, Mingkui Wei, and Junjie Xiong. 2026. [Prompt overflow: What the guardrail inspects is not what the model infers](#). *Preprint*, arXiv:2605.23196.

Appendix

A.1 The Challenges of Single-Objective Techniques Against Guardrail Systems

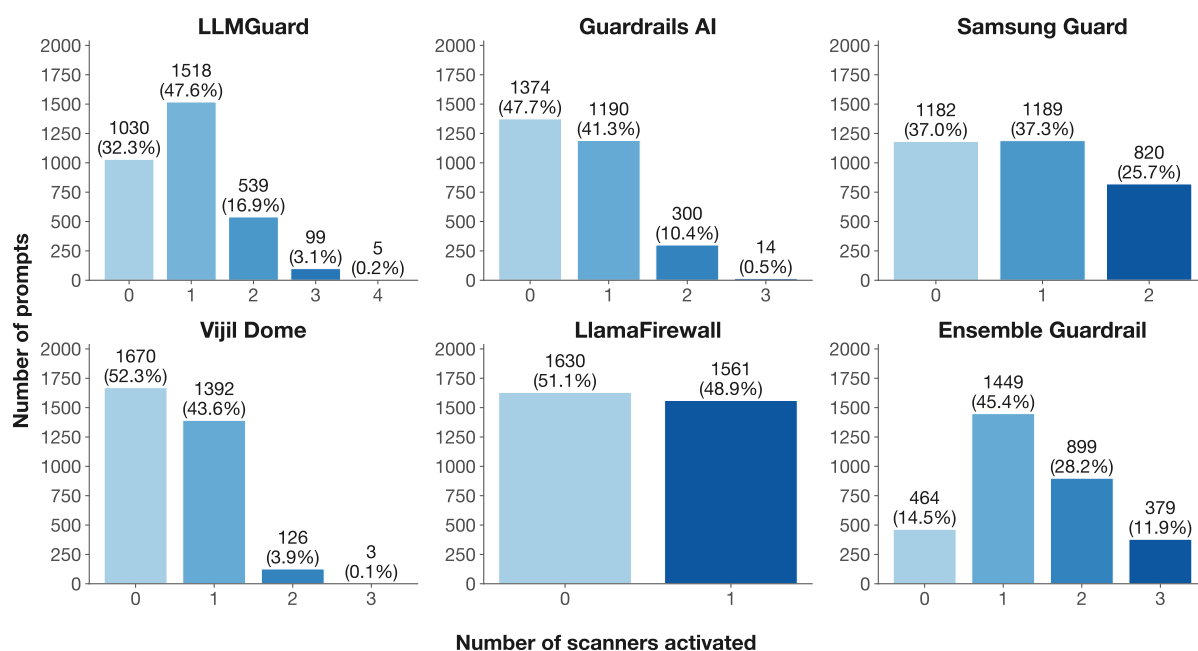


Figure A.1: **Number of Scanners Activated per Prompt, Across all Targets.** Distribution of how many scanners simultaneously flag each of the 3,191 PIGuard candidate prompts, per target guardrail system.

A.2 Experimental Setup

Table A.1: **Perturbation Techniques.** All 11 perturbation techniques used in Exhaustive and BRANCH.

Evasion Attack	Description
TextFooler (Jin et al., 2020)	Words with the highest importance ranking are replaced with suitable replacement words with similar semantic meaning.
DeepWordBug (Gao et al., 2018)	Character-level transformations are applied to the highest-ranked tokens to minimise distance of the perturbation.
TextBugger (Li et al., 2019)	Generates utility-preserving adversarial text against black-box and white-box classification systems.
PWWS (Ren et al., 2019)	Probability Weighted Word Saliency (PWWS) ranks word importance using word saliency and classification probability.
Pruthi (Pruthi et al., 2019)	Generates perturbations in the form of adversarial spelling mistakes via removing or swapping characters.
PSO (Zang et al., 2020)	Combines word substitution with particle swarm optimization (PSO) to search for adversarial candidates.
BAE (Garg and Ramakrishnan, 2020)	Contextual perturbations from a BERT-MLM masked model by replacing and inserting masked tokens in the prompt.
CLARE (Li et al., 2021)	Uses a pretrained masked language model to replace, insert, and merge tokens via a greedy search, constrained by sentence-embedding similarity.
IGA (Wang et al., 2021)	An improved genetic algorithm (GA) that swaps words with their embedding nearest-neighbours.
Faster Alzantot (Jia et al., 2019)	A faster variant of Alzantot’s genetic algorithm (GA) word-embedding substitution, using a smaller language model.
Kuleshov (Kuleshov et al., 2018)	Swaps words based upon opposite embeddings.

Table A.2: **Experiment Target Detector Categories.** Highlights what each guardrail system within our experimentation is designed to detect and block. Categories vary across targets based on capability and implemented scanners.

Detector category	LLM Guard	Vijil Dome	Guardrails AI	Samsung Guard	LlamaFirewall	Ensemble Guardrail
Prompt Injection	✓	✓	✗	✗	✓	✓
Jailbreak	✗	✗	✓	✓	✓	✓
Toxicity	✓	✓	✓	✗	✗	✓
Gibberish	✓	✗	✓	✗	✗	✓
Harm	✗	✗	✗	✓	✗	✗
Moderation	✗	✗	✗	✓	✗	✗
Adversarial Suffix	✓	✓	✗	✗	✗	✗
Malicious Code	✗	✗	✗	✗	✓	✗
Invisible Text	✓	✓	✗	✗	✗	✗
Non-English Language	✓	✗	✗	✗	✗	✗
Code	✓	✗	✗	✗	✗	✗
Total Categories	7	4	3	3	3	4

A.3 Query Efficiency and Technique Composition

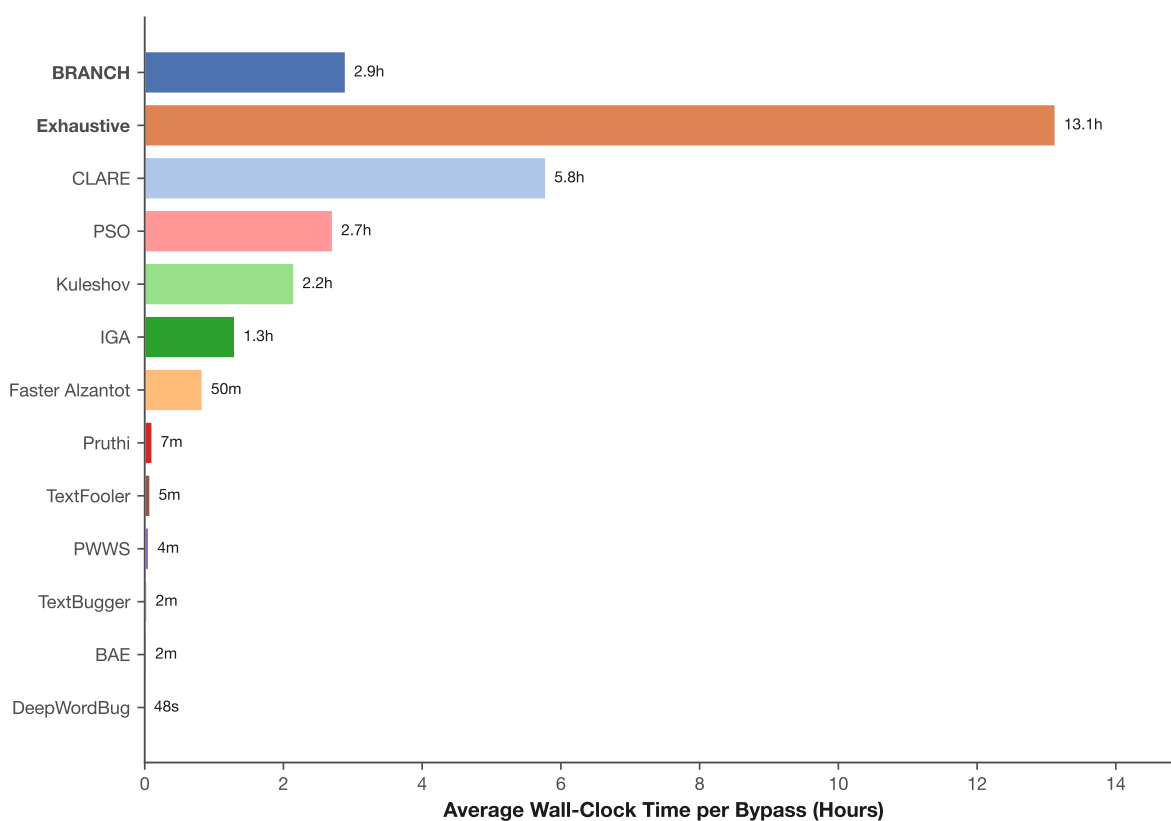


Figure A.2: **Average Wall-Clock Time by Method.** Average wall-clock time per bypass job for BRANCH, Exhaustive, and the 11 individual techniques, averaged across all 6 target guardrail systems.

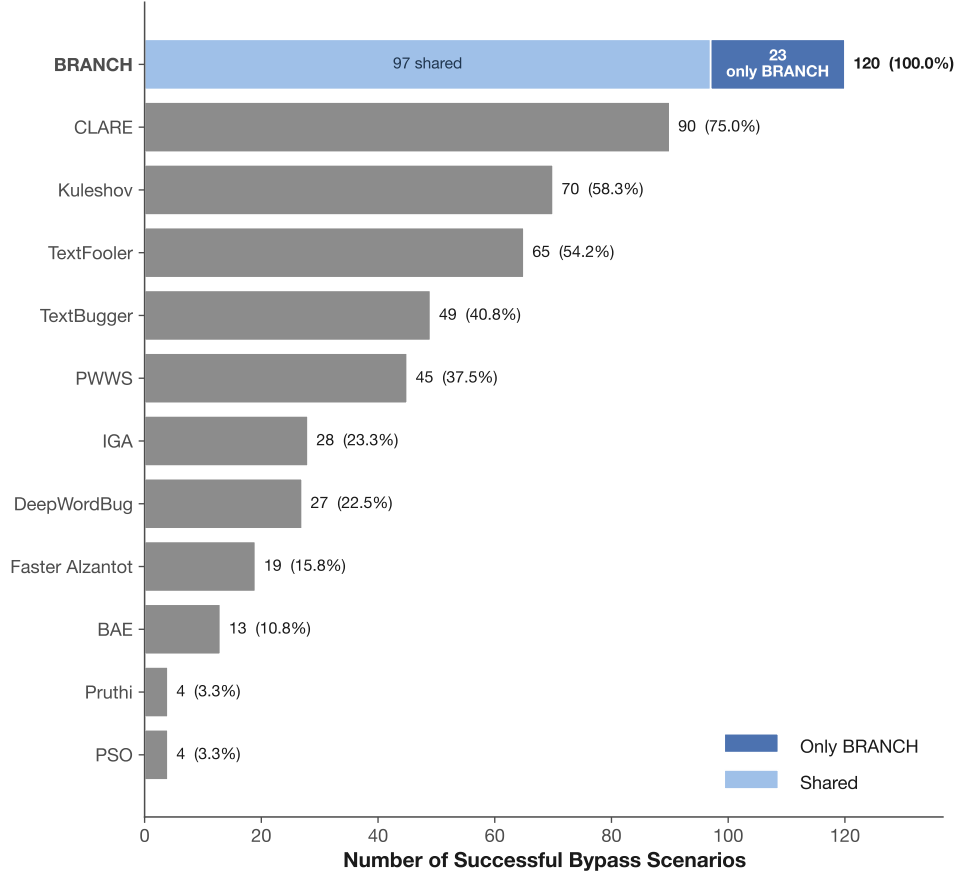


Figure A.3: **BRANCH vs Individual Techniques Coverage.** Per-prompt bypass coverage, showing the effectiveness of each technique across different guardrail systems.

A.4 Bypass Similarity

Table A.3: **Semantic Preservation and Attack Success Rate per Technique and Target.** n is the number of successful bypasses the statistics are computed over, *Pres.* the average semantic preservation ± 1 standard deviation, and the ASR over the 20 seed prompts.

Method	Guardrails AI			LLM Guard			LlamaFirewall			Samsung Guard			Vijil Dome			Ensemble Guardrail		
	n	Pres.	ASR	n	Pres.	ASR	n	Pres.	ASR	n	Pres.	ASR	n	Pres.	ASR	n	Pres.	ASR
BRANCH	68	0.65 ± 0.17	100	75	0.86 ± 0.10	100	97	0.81 ± 0.12	100	56	0.83 ± 0.11	100	65	0.81 ± 0.11	100	53	0.72 ± 0.16	100
Exhaustive	62	0.81 ± 0.13	90	78	0.87 ± 0.10	85	112	0.82 ± 0.11	95	75	0.85 ± 0.11	95	78	0.84 ± 0.10	90	6	0.76 ± 0.09	30
BAE	3	0.90 ± 0.05	15	2	0.95 ± 0.02	10	3	0.90 ± 0.09	15	2	0.97 ± 0.03	10	3	0.88 ± 0.11	15	–	–	–
CLARE	18	0.73 ± 0.15	90	17	0.89 ± 0.08	85	17	0.86 ± 0.06	85	19	0.91 ± 0.06	95	14	0.89 ± 0.09	70	5	0.75 ± 0.10	25
DeepWordBug	5	0.79 ± 0.14	25	5	0.83 ± 0.14	25	9	0.74 ± 0.19	45	3	0.65 ± 0.19	15	5	0.81 ± 0.09	25	–	–	–
Faster Alzantot	2	0.94 ± 0.02	10	3	0.91 ± 0.10	15	6	0.79 ± 0.08	30	5	0.89 ± 0.03	25	3	0.84 ± 0.09	15	–	–	–
IGA	4	0.84 ± 0.09	20	5	0.85 ± 0.07	25	8	0.85 ± 0.07	40	6	0.88 ± 0.05	30	5	0.86 ± 0.09	25	–	–	–
Kuleshov	8	0.90 ± 0.06	40	14	0.86 ± 0.12	70	17	0.82 ± 0.13	85	15	0.84 ± 0.08	75	15	0.86 ± 0.08	75	1	0.81 ± 0.00	5
Pruthi	1	0.92 ± 0.00	5	2	0.94 ± 0.03	10	1	0.94 ± 0.00	5	–	–	–	–	–	–	–	–	–
PSO	–	–	–	2	0.95 ± 0.02	10	–	–	–	1	0.83 ± 0.00	5	1	0.90 ± 0.00	5	–	–	–
PWWS	6	0.82 ± 0.07	30	7	0.90 ± 0.05	35	15	0.80 ± 0.11	75	8	0.87 ± 0.09	40	9	0.81 ± 0.06	45	–	–	–
TextBugger	9	0.79 ± 0.11	45	8	0.86 ± 0.10	40	17	0.81 ± 0.10	85	7	0.76 ± 0.13	35	8	0.83 ± 0.09	40	–	–	–
TextFooler	7	0.86 ± 0.06	35	13	0.83 ± 0.10	65	19	0.85 ± 0.10	95	11	0.84 ± 0.13	55	15	0.78 ± 0.14	75	–	–	–

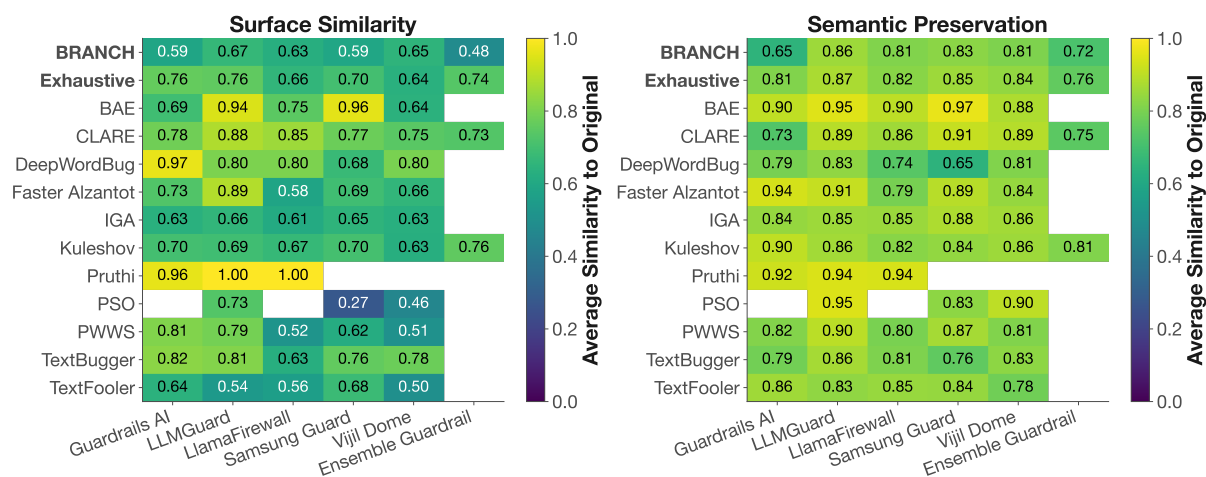


Figure A.4: **Average Surface Similarity and Semantic Preservation to the Original Seed Prompt.** Exact per-target values for BRANCH, Exhaustive, and each of the 11 individual techniques.

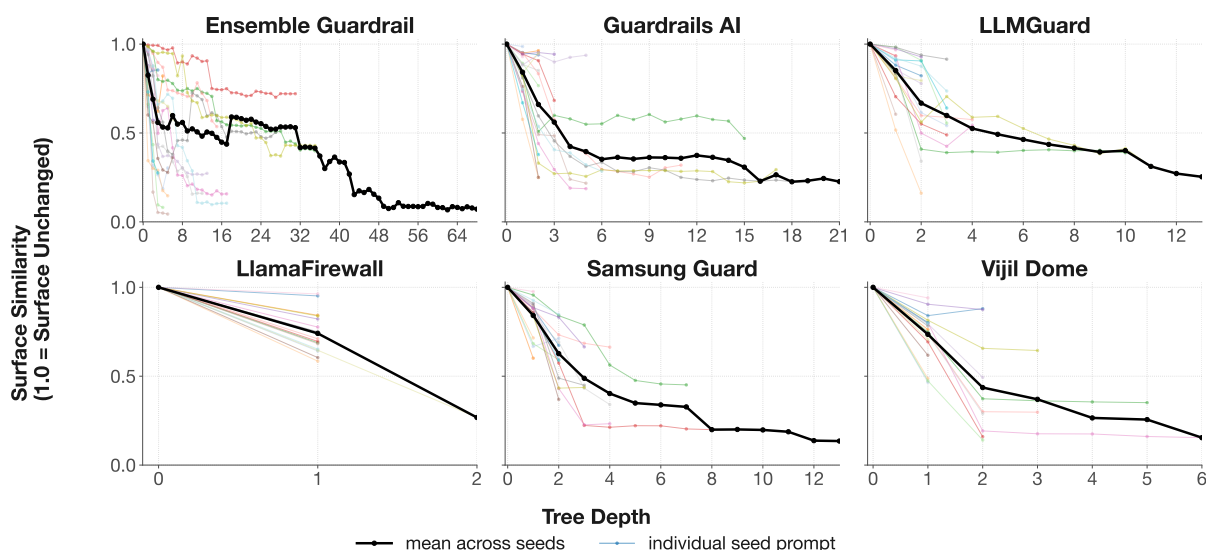


Figure A.5: **BRANCH Surface Similarity.** Shows recorded surface similarity at each depth across all bypass scenarios against all targets.

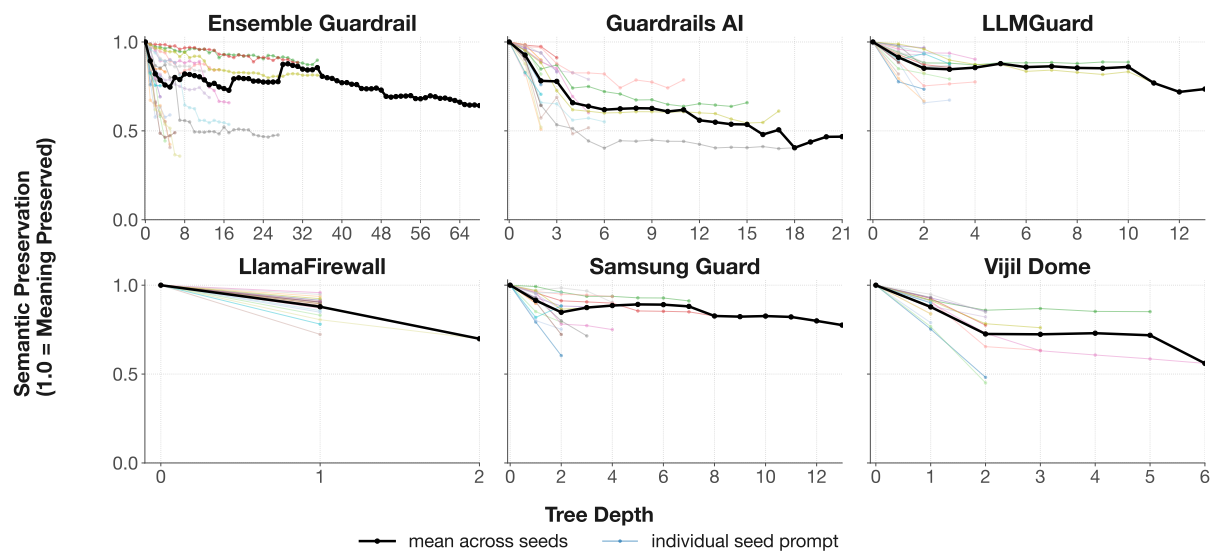


Figure A.6: **BRANCH Semantic Preservation.** Shows recorded semantic preservation at each depth across all bypass scenarios against all targets.

A.5 Transferability

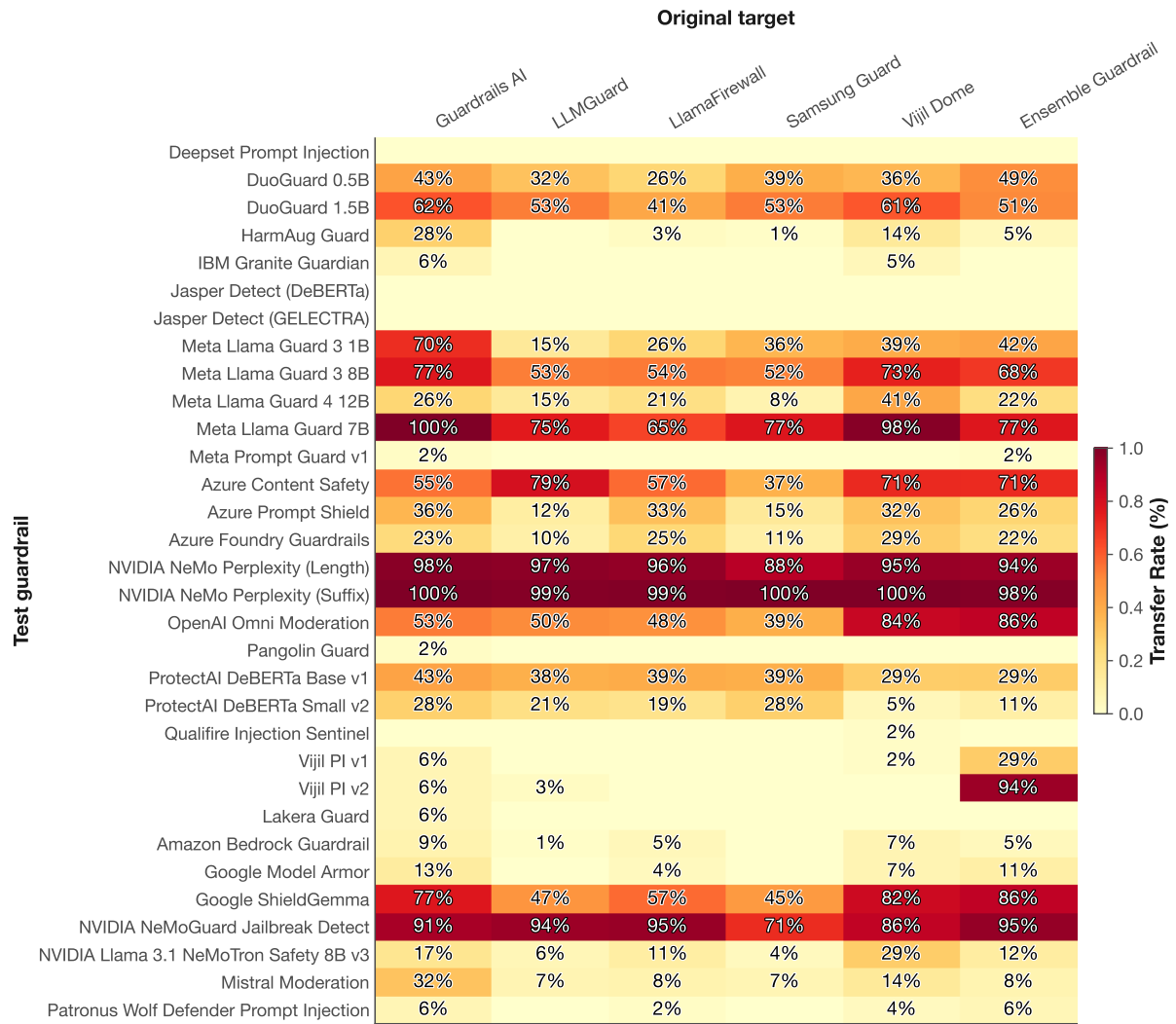


Figure A.7: **Transferability Heatmap for BRANCH.** Shows percentage of generated bypasses from the original target (x) transferring to an unseen guardrail (y).